Instituto Superior de Engenharia do Porto

Licenciatura em Engenharia Informática (LEI)

Laboratório de Projeto (LAPR) - Projeto Integrador

GraphCode-Reviewer: Sistema de Auditoria de Software Semântica e Multiagente Baseado em Grafos de Propriedade de Código (CPG) e Model Context Protocol (MCP) com Kotlin

1. Enquadramento e Integração Curricular (LEI-ISEP)

O ensino de Engenharia Informática no ISEP assenta no modelo CDIO (Conceber, Desenhar, Implementar, Operar).[1] No segundo e terceiro anos do plano de estudos, os estudantes são expostos a projetos de integração multidisciplinar (LAPR).[2] O **GraphCode-Reviewer** foi concebido de raiz para constituir o culminar prático desta aprendizagem, obrigando o estudante a resgatar e unificar de forma sinérgica as seguintes competências disciplinares [5]:

- **EAPLI (Conceção de Software - 2º Ano):** Aplicação estrita de Domain-Driven Design (DDD), isolando regras de negócio em Agregados, Entidades e Serviços de Domínio puros.[3] A persistência no Neo4j é modelada com Spring Data Neo4j (SDN) usando tipagem forte em Kotlin.[6]
- **ESINF (Estruturas de Informação - 2º Ano):** Análise e manipulação de estruturas não lineares (grafos).[8] Implementação de algoritmos concorrentes de travessia (como Dijkstra/Shortest Path) para análise de impacto semântico de alterações de código.[9]
- **RCOMP (Redes de Computadores - 2º Ano):** Implementação de um servidor Model Context Protocol (MCP) assíncrono recorrendo a sockets TCP concorrentes, comunicações JSON-RPC e gestão estrita de timeouts e controlo de fluxo.[11]
- **Compiladores e Gramáticas (3º Ano):** Parsing sintático incremental através da integração de bindings Kotlin/Java para o Tree-sitter, convertendo código estruturado numa Árvore de Sintaxe Abstrata (AST).[13]
- **LAPR5 (Laboratório de Projeto 3 - 3º Ano):** Construção de uma Single Page Application (SPA) reativa em TypeScript e React para renderização gráfica e navegação interativa tridimensional na topologia do Grafo de Propriedades de Código.[8]

A Tabela 1 detalha os componentes e marcos administrativos do projeto para o semestre letivo:

Tabela 1: Metadados e Planeamento Administrativo do LAPR

Parâmetro de Projeto	Detalhe e Especificação Académica
Sigla do Projeto	GraphCode-Reviewer
Unidade Curricular	Laboratório de Projeto (LAPR)
Metodologia de Gestão	Scrum Iterativo e Incremental
Duração Total	12 semanas de desenvolvimento ativo
Frequência de Sprints	3 Sprints de 4 semanas de duração
Linguagem Backend	Kotlin 2.1 (JVM 21)
Linguagem Frontend	TypeScript 5.4 / React 18
Persistência Base	Neo4j Graph Database (v5.x Enterprise)
Parser Sintático	Tree-sitter Runtime Bindings

2. Descrição do Problema e Inovação Técnico

A integração clássica de modelos de linguagem (LLMs) nas pipelines de Integração Contínua (CI) é financeiramente insustentável. Ferramentas que empacotam o repositório como texto plano (como o Repomix) provocam o fenómeno de *Context Window Bloat*.[17] O envio indiscriminado de milhares de linhas de código utilitário e boilerplate satura cognitivamente os LLMs devido ao problema *Lost in the Middle*, gerando elevados tempos de resposta e uma proliferação inaceitável de falsos negativos em auditorias de segurança.[3]

A inovação proposta assenta no paradigma de **Context Engineering estruturado**.[18] Em vez de submeter ficheiros de texto completos, o sistema analisa localmente o repositório utilizando o parser incremental Tree-sitter.[12] O código é convertido numa AST e persistido no Neo4j como um **Grafo de Propriedades de Código (Code Property Graph - CPG)**, mapeando classes, funções, endpoints de API e as suas inter-relações lógicas (chamadas, heranças, injeção de dependências).[9]

Este grafo de conhecimento é exposto de forma bidirecional a agentes de IA locais e remotos

através do **Model Context Protocol (MCP)**.[20] O assistente de IA deixa de ler ficheiros sequencialmente; em vez disso, executa chamadas de ferramentas (*tool calls*) padronizadas que disparam queries Cypher otimizadas.[21] O impacto prático é uma **redução documentada de até 99.2% no consumo de tokens de entrada** em projetos de média escala (passando de $\sim$ tokens numa exploração exaustiva de ficheiros para apenas $\sim$ tokens via consultas MCP).[12]

A Tabela 2 compara tecnicamente o posicionamento do GraphCode-Reviewer face às alternativas de mercado:

Tabela 2: Posicionamento Tecnológico e Comparativo de Alternativas

Dimensão Técnica	Abordagem Clássica (Grep/Repomix)	Plataformas Genéricas (Aider Map)	Proposta ISEP: GraphCode-Reviewer
Soberania do Código	Baixa (envio total para cloud pública) [3]	Média (envio parcial de ficheiros)	Máxima (perímetro local / Azure AI) [3]
Estrutura de Dados	Texto bruto ou XML desestruturado [22]	Árvores e tags de proximidade [22]	Grafo de Propriedades de Código (Neo4j) [9]
Tipo de Comunicação	Carga manual de ficheiros inteiros	Endpoints proprietários e efémeros	Protocolo Padrão Aberto (MCP via JSON-RPC) [13]
Consumo de Tokens	Elevado (escala com o tamanho dos ficheiros) [3]	Moderado (mapa simplificado de assinaturas)	Minimalista (redução de 99.2% comprovada) [12]
Lógica de Decisão	Probabilística (LLM aprova/rejeita o código)	Probabilística (o agente edita diretamente)	Determinística (regras de domínio em Kotlin) [3]

3. Arquitetura e Modelo de Dados do Grafo (Neo4j)

O backend do sistema é desenvolvido em Kotlin aplicando os conceitos de **Clean Architecture** e **Ports & Adapters**.[23] O domínio puro (core) é hermeticamente isolado de bibliotecas externas, permitindo testes unitários e de integração locais sem dependências de infraestrutura.[3]

O Grafo de Propriedades de Código gerado pelo Tree-sitter e armazenado no Neo4j segue um esquema rigoroso de nós e relações que abstrai a topologia do software.[9] A Tabela 3 apresenta o mapeamento dos nós e arestas do CPG:

Tabela 3: Ontologia e Estrutura do Grafo de Propriedades de Código

Tipo de Elemento	Rótulo / Label do Elemento	Atributos e Propriedades Relevantes	Função Semântica no Grafo de Código
Nó	File	file_path, content_hash, size_bytes [12]	Representa um ficheiro físico no repositório.[12]
Nó	Class	name, qualified_name, is_exported [9]	Mapeia declarações de classes e tipos abstratos.[9]
Nó	Function	name, signature, return_type, complexity [9]	Representa métodos, subrotinas ou funções.[9]
Nó	Endpoint	method, path, handler_name [9]	Mapeia rotas expostas públicas de APIs (REST).[9]
Nó	DataModel	name, fields, schema_type [9]	Representa estruturas de dados persistidas ou DTOs.[9]
Nó	Test	type, suite_name, is_ignored [9]	Identifica componentes de verificação automatizada.[9]
Aresta	CONTAINS	Nenhuma	Relação de pertença (ex: File contém Class).[9]

Aresta	CALLS	line_number, is_async [9]	Relação direcionada de invocação entre funções.[9]
Aresta	HANDLER	binding_route [9]	Liga um Endpoint público à Function que o processa.[9]
Aresta	IMPLEMENTS	Nenhuma	Conecta uma Class ao contrato/interface que realiza.[9]
Aresta	PARENT_OF	Nenhuma	Representa a hierarquia de herança entre classes.[9]

4. Planeamento do Projeto Integrador

A execução do projeto é estruturada em 3 Sprints com foco incremental, assegurando que o estudante desempenhe papéis ágeis de engenharia.[11] A Tabela 4 apresenta o planeamento geral dos Sprints:

Tabela 4: Mapeamento de Sprints e Foco Curricular

Sprint ID	**Foco Tecnológico e Arquitetural**	**Unidades Curriculares Principais**	**Entregas e Artefactos de Engenharia**
Sprint 1	Parser de Código & Grafo DDD	Compiladores, EAPLI, ESINF	Parser Tree-sitter em Kotlin, BD Neo4j e injeção CPG.[9]
Sprint 2	Servidor MCP & Redes TCP	RCOMP, ESINF, EAPLI	Servidor JSON-RPC TCP, ferramentas MCP e Cypher.[20]
Sprint 3	Webhooks, GRC & Consola SPA	LAPR4, LAPR5	Gateway assíncrono, motor determinístico e UI

			3D.[8]

5. Caderno de Encargos (User Stories e Critérios de Aceitação)

Nota: Conforme as especificações de legibilidade do ISEP, nenhuma User Story (US) ou Critério de Aceitação (AC) está inserido em tabelas, permitindo uma visualização fluida e estruturada para exportação em PDF.

Sprint 1: Parser de Código, Modelo DDD e Persistência em Grafo (LAPR/EAPLI/ESINF)

US 1.1 - Parser de Código Incremental com Tree-sitter em Kotlin

Como sistema, quero invocar e consumir a biblioteca Tree-sitter no backend Kotlin para processar ficheiros de código do repositório de forma incremental e gerar a Árvore de Sintaxe Abstrata (AST) correspondente.[12]

Critérios de Aceitação (AC):

- **AC 1.1.1:** O parser deve carregar de forma nativa e segura os bindings JNI do Tree-sitter (como java-tree-sitter) sem quebrar a execução da JVM.[24]
- **AC 1.1.2:** A análise de código deve identificar de forma determinística heranças de classes, assinaturas de funções, tipos de retorno, decoradores e endpoints de API.[9]
- **AC 1.1.3:** Cada nó extraído pelo Tree-sitter deve reter a sua posição física exata no ficheiro original (start_line, end_line, start_byte, end_byte) para enriquecimento posterior do relatório de code review.[12]

US 1.2 - Modelação e Persistência do CPG no Neo4j com Spring Data

Como programador, quero modelar a persistência do Grafo de Propriedades de Código (CPG) no Neo4j usando classes de domínio Kotlin e JPA/OGM, respeitando as regras de isolamento de Domain-Driven Design (DDD).[26]

Critérios de Aceitação (AC):

- **AC 1.2.1:** A modelação do domínio do código deve ser definida com tipagem estática pura em Kotlin, utilizando annotations do Spring Data Neo4j (como @Node e @Relationship).[6]
- **AC 1.2.2:** O sistema deve implementar transações ACID consistentes na base de dados Neo4j, geridas a partir de repositórios de domínio que estendem Neo4jRepository.[6]
- **AC 1.2.3:** A persistência deve ser incremental: o sistema deve ler os *hashes* de conteúdo de cada ficheiro e reanalisar apenas os subgrafos de ficheiros que sofreram modificações desde a última análise, prevenindo reescritas totais.[12]

US 1.3 - Motor Estático Determinístico em Kotlin (Anti-Vulnerabilidade)

Como pipeline, quero executar um motor estático de regras locais em Kotlin para intercetar

vulnerabilidades básicas óbvias sem necessidade de invocar APIs externas de IA de forma probabilística.[3]

Critérios de Aceitação (AC):

- **AC 1.3.1:** O motor local deve analisar os ficheiros de código utilizando expressões regulares compiladas de alto desempenho para detetar segredos expostos (*hardcoded secrets*), desativação de validação SSL/TLS e configurações CORS perigosas.[3]
- **AC 1.3.2:** A execução do motor estático local deve preceder qualquer chamada a APIs de IA, injetando os seus resultados diretamente no domínio para poupança de tokens em chamadas redundantes.[3]
- **AC 1.3.3:** O motor estático deve permitir a injeção assíncrona de ficheiros locais de governação do próprio repositório (como .codereview.yml) para adaptar as regras de validação por equipa.[3]

Sprint 2: Protocolo de Comunicação e Exposição de Ferramentas MCP (RCOMP/ESINF)

US 2.1 - Servidor MCP Assíncrono com JSON-RPC em Kotlin

Como cliente de IA (como Cursor, Claude Code ou Windsurf), quero conectar-me a um servidor Model Context Protocol (MCP) assíncrono em Kotlin via sockets TCP concorrentes para aceder aos dados estruturados do repositório.[13]

Critérios de Aceitação (AC):

- **AC 2.1.1:** O servidor MCP deve implementar estritamente a especificação oficial do protocolo, processando mensagens assíncronas no formato JSON-RPC sobre canais estáveis (stdio ou HTTP seguro).[12]
- **AC 2.1.2:** O backend em Kotlin deve gerir concorrência não-bloqueante recorrendo a *Coroutines* do Kotlin, garantindo que sessões concorrentes de múltiplos utilizadores ou agentes são isoladas e mantêm latência mínima de rede.[3]
- **AC 2.1.3:** O tempo de resposta de rede do servidor para o handshake inicial de negociação do protocolo MCP não deve exceder os 100 milissegundos.[13]

US 2.2 - Exposição de Ferramentas Estruturadas do Grafo de Código

Como agente de IA, quero consumir ferramentas MCP estruturadas do GraphCode-Reviewer para navegar semântica e topologicamente pelas dependências e estruturas do código.[9]

Critérios de Aceitação (AC):

- **AC 2.2.1:** O servidor MCP deve expor as seguintes ferramentas estruturadas com esquemas JSON estritos: stakgraph_search (procura vetorial), stakgraph_map (relação de vizinhança) e stakgraph_code (obtenção de código).[9]
- **AC 2.2.2:** O sistema de consultas deve expor uma ferramenta para o cálculo do caminho mínimo (Shortest Path) entre dois nós quaisquer do grafo, usando algoritmos de ESINF para detetar a cadeia completa de impacto de uma alteração.[9]
- **AC 2.2.3:** A ferramenta de leitura de código (stakgraph_code) deve retornar

exclusivamente os subgrafos solicitados, impedindo o envio de ficheiros adjacentes e mitigando o desperdício de tokens.[9]

US 2.3 - Tradutor Semântico de Linguagem Natural para Cypher

Como agente MCP, quero submeter perguntas lúdicas em linguagem natural e receber os dados correspondentes da base de dados através de uma tradução dinâmica para a linguagem de consulta Cypher.[12]

Critérios de Aceitação (AC):

- **AC 2.3.1:** O tradutor semântico do servidor MCP deve converter pedidos textuais (ex: "quem chama a função x?") na instrução de query Cypher exata necessária para ler o Neo4j.[30]
- **AC 2.3.2:** A conversão deve ser guiada por esquemas lógicos e restrições estruturais (*schema-guided extraction*) passados dinamicamente do Neo4j para o modelo de tradução, eliminando alucinações de sintaxe.[23]
- **AC 2.3.3:** Caso uma query Cypher resulte em falha sintática no Neo4j, o servidor deve intercetar a exceção de forma segura e devolvê-la ao tradutor para auto-correção iterativa automática.[23]

Sprint 3: Integração DevOps, Governação Determinística e Visualização (LAPR4/LAPR5)

US 3.1 - Gateway de Webhooks com Deduplicação Distribuída via Redis

Como servidor de auditoria, quero expor um endpoint HTTP REST seguro (Webhook Gateway) em Kotlin para receber e processar eventos concorrentes de Pull Requests originados no Azure DevOps.[3]

Critérios de Aceitação (AC):

- **AC 3.1.1:** O gateway de receção de webhooks deve processar de forma assíncrona os eventos do Azure DevOps (git.pullrequest.created, git.pullrequest.updated), garantindo o envio do estado HTTP 202 Accepted ao cliente em menos de 10 segundos, contornando o limite síncrono da plataforma.[3]
- **AC 3.1.2:** O gateway deve validar a autenticidade dos webhooks recebidos através de assinaturas criptográficas HMAC-SHA256 geradas a partir de um segredo partilhado, rejeitando requisições fraudulentas de forma célere.[3]
- **AC 3.1.3:** O sistema deve integrar um mecanismo de deduplicação distribuída via Redis com TTL e bloqueio exclusivo concorrente (*distributed lock*), garantindo que o mesmo evento concorrente de PR não despoleta análises redundantes em instâncias de servidores escalados horizontalmente.[7]

US 3.2 - Motor de Governação Determinística e Publicação de Relatórios

Como pipeline de CI/CD, quero que os veredictos finais de fusão de código (merge gates) sejam governados por regras de domínio fixas e determinísticas baseadas nos relatórios analíticos, impedindo que uma aprovação seja delegada de forma direta a modelos de IA

probabilísticos.[3]

Critérios de Aceitação (AC):

- **AC 3.2.1:** O veredicto final deve ser determinado por regras Kotlin rígidas a partir de pontuações ponderadas calculadas com base nos achados de segurança extraídos do grafo (Critical, High, Medium, Low).[3]
- **AC 3.2.2:** A interceção de qualquer vulnerabilidade grave (Critical) deve acionar automaticamente uma barreira de integridade rígida (MUST gate), forçando o veredicto de bloqueio do Pull Request lido.[3]
- **AC 3.2.3:** O sistema deve publicar os resultados das análises sob a forma de comentários Markdown formatados de forma progressiva diretamente nas linhas de alteração afetadas do Pull Request no Azure DevOps.[3]

US 3.3 - Consola Web Gráfica de Visualização Tridimensional do CPG (LAPR5)

Como estudante ou revisor de código, quero utilizar uma consola gráfica interativa baseada na web (Single Page Application - SPA) desenvolvida em TypeScript para visualizar e interagir tridimensionalmente com a estrutura semântica de código do meu repositório.[8]

Critérios de Aceitação (AC):

- **AC 3.3.1:** A consola gráfica web deve ser construída como uma SPA em TypeScript (recorrendo a bibliotecas como React e NVL/D3 para renderização tridimensional), garantindo tempos de carregamento conformes com os requisitos de LAPR5.[8]
- **AC 3.3.2:** A consola gráfica deve renderizar interativamente a topologia completa do Grafo de Propriedades de Código, evidenciando de forma visual quais os nós (classes, funções, endpoints) que foram alterados na submissão ativa e os caminhos lógicos potencialmente impactados por regressão.[23]
- **AC 3.3.3:** O utilizador deve poder clicar em qualquer nó do grafo exibido na interface gráfica para abrir uma janela de visualização do código-fonte correspondente ou para despoletar de forma interativa a análise de caminho mínimo (*shortest path*) até outro componente.[12]

6. Modelo de Avaliação e Métricas de Validação

A validação de desempenho do sistema segue abordagens experimentais rigorosas em engenharia de software.[3] Com vista a calcular taxas de falsos negativos com precisão, a avaliação recorre a testes controlados constituídos por defeitos conhecidos e catalogados introduzidos manualmente nos repositórios de teste.[3]

As métricas de classificação binária são modeladas a partir das seguintes formulações matemáticas:

Precisão (Precision)

Mede o rácio de acuidade do sistema, representando a proporção de alertas emitidos pela ferramenta que correspondem a anomalias de segurança ou código reais e confirmadas [3]:

$$\text{Precision} = \frac{\text{VP}}{\text{VP} + \text{FP}}$$

Onde VP designa os Verdadeiros Positivos (defeitos de código reportados corretamente pelo sistema) e FP os Falsos Positivos (alertas de anomalia errados gerados pelo modelo de linguagem).[3]

Revocação (Recall)

Mede o rácio de sensibilidade analítica do sistema, avaliando a proporção de defeitos lógicos, vulnerabilidades sintáticas e falhas funcionais injetados que foram intercetados pelo pipeline [3]:

$$\text{Recall} = \frac{\text{VP}}{\text{VP} + \text{FN}}$$

Onde FN representa os Falsos Negativos (erros injetados e catalogados que passaram na auditoria sem gerar alerta do sistema).[3]

F1-Score (F_1)

Constitui a média harmónica entre as duas métricas anteriores, servindo como o indicador unificado de robustez analítica do classificador:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

A meta de sucesso estipulada para a validação da prova de conceito exige que o sistema atinja um $F_1 \geq 0,80$ em ambientes de teste de média e larga escala (Medium/Massive), determinando adicionalmente que a taxa de deteção para anomalias consideradas severas ou críticas seja absoluta ($\text{Recall}_{\text{Critical}} = 1,00$).[3]

A eficiência operacional, a latência de processamento assíncrono e a poupança económica induzida pela injeção contextual baseada no Grafo de Propriedades de Código são avaliadas contra os seguintes valores e objetivos experimentais corporativos:

Tabela 5: Foco de Métricas Operacionais e Metas de Desempenho

Indicador de	Brute-force (ex:	Grafo Semântico +	Meta de Sucesso

Eficiência e Performance	Repomix)	MCP (GraphCode-Reviewer)	de LAPR
Consumo Médio de Tokens de Entrada	$\sim$ tokens por PR [12]	$\sim$ tokens por PR [12]	**Redução de tokens** $\geq$
Custo de Inferência por Pull Request	$\sim$ por análise [3]	$\sim$ por análise [3]	**Custo por PR** $<$ [3]
Latência Completa de Processamento	$127,5$ segundos de pipeline [3]	$55,3$ segundos de pipeline [3]	**Tempo total de pipeline** $<$ **s** [3]
Sensibilidade semântica (Recall Global)	25% (modelo monolítico saturado) [3]	88% (arquitetura multiagente) [3]	**Recall global** $\geq$ [3]
Acuidade sem Ruído (Precisão Global)	76% (falsos positivos frequentes) [3]	96% (motor de regras + CoT) [3]	**Precisão global** $\geq$

7. Conclusão e Diretrizes de Engenharia

O desenho e o enunciado do projeto **GraphCode-Reviewer** estabelecem um desafio pedagógico completo e rigoroso para os anos finais da Licenciatura em Engenharia Informática no ISEP.[1] Ao convergir os princípios clássicos de análise léxico-gramatical e parsing de compiladores com a representation estrutural avançada de grafos, os estudantes elevam o seu nível de competência técnica para patamares industriais reais, abandonando a criação de protótipos redundantes em sandbox académica.[4]

A centralidade dada ao Grafo de Propriedades de Código (CPG) e ao Model Context Protocol (MCP) resolve diretamente os problemas reais de orçamento e latência que limitam o uso de inteligência artificial em contexto de produção contínua de software.[12] Ao blindar a natureza probabilística de Large Language Models através de uma camada de regras determinísticas rigorosas e análise de fluxo de dados, o projeto demonstra com factos experimentais que a inteligência artificial só atinge a máxima precisão quando governada e orientada por boas práticas de Engenharia de Software.[13]

8. Grelha de Avaliação e Critérios de Correção

Rigorosos

Para garantir o máximo rigor na avaliação e impedir a atribuição facilitada de notas de topo (17-20 valores), a correção do projeto é governada por uma grelha multidimensional estrita. A nota final é calculada por uma formulação matemática ponderada das classificações obtidas em cinco vertentes de engenharia [3]:

$$\text{NotaFinal} = \sum_{i=1}^{5} (\text{Classificação}_i \times \text{Peso}_i)$$

A Tabela 6 estabelece os pesos atribuídos a cada componente de avaliação:

Tabela 6: Distribuição de Pesos para Classificação Final do Projeto

ID	Componente Multidimensional de Avaliação	Peso na Nota Final
C1	Conceção de Software e Rigor Arquitetural (EAPLI / DDD / Ports & Adapters)	25%
C2	Qualidade de Código, Concorrência e Parsing (Kotlin Coroutines / Tree-sitter / Neo4j)	30%
C3	Cobertura e Qualidade da Suíte de Testes (Unitários / Integração / SDN Testcontainers)	15%
C4	Eficiência de Contexto, Sizing e GRC (Poda / Token-Saving / Veredicto Determinístico)	15%
C5	Interface de Utilizador e Visualização 3D do CPG (SPA TypeScript / NVL /	15%

	React)	

8.1 Critérios de Correção Detalhados por Intervalo de Desempenho

Nível 5: Excelente / Excecional (19 a 20 Valores)

Reservado exclusivamente a projetos com maturidade de engenharia de nível industrial. Todos os seguintes critérios devem ser cumpridos sem exceção:

- **Isolamento Absoluto do Core:** Zero importações de bibliotecas de infraestrutura (como httpx, redis, Spring Boot ou Ktor) na camada de domínio puro. Qualquer violação deste limite reduz imediatamente a classificação desta vertente (C1) para o patamar inferior.[3]
- **Rigor Concorrente:** Ausência total de bloqueios síncronos na leitura do sistema de ficheiros ou processamento de rede. O estudante deve implementar e provar o uso de Kotlin Coroutines assíncronas com tratamento concorrente real via async e awaitAll para processamento paralelo.[3]
- **Suíte de Testes Excecional:** Cobertura global de código verificada por JaCoCo ou pytest-cov superior ou igual a 90%, atingindo obrigatoriamente 100% de cobertura nos componentes críticos de fronteira, nomeadamente: autenticação criptográfica de webhooks, motor de pruning e normalização de veredicto.[3]
- **Resiliência a Concorrência Reativa:** Implementação funcional e comprovada de um distributed lock via Redis (SET NX EX) no webhook gateway, impedindo de forma absoluta a execução duplicada de análises para Pull Requests sob condições de concorrência ativa.[3]
- **CPG Integrado e MCP Operacional:** O servidor MCP deve processar queries estruturadas JSON-RPC com latência de handshake inferior a 100 milissegundos, gerando respostas válidas Cypher sem alucinação de sintaxe no Neo4j.[13]
- **Visualização SPA 3D Completa:** A consola em TypeScript renderiza em tempo real um grafo de propriedades interativo em 3D usando a biblioteca NVL. No ecrã, é visualmente evidente o destaque em vermelho do caminho de impacto de uma alteração de código, calculado deterministicamente por algoritmos de shortest-path.[8]
- **Apresentação Científica:** O estudante deve discutir de forma crítica na defesa os limites da análise estática pura e documentar o fenómeno da "Ilusão da Assinatura" verificado empiricamente no seu projeto.[3]

Nível 4: Muito Bom (17 a 18 Valores)

Projetos com solidez técnica exemplar que falham ligeiramente em otimizações extremas ou detalhes de cobertura de testes:

- **Isolamento de Domínio Funcional:** O isolamento de DDD e Arquitetura Hexagonal está presente, contudo, observam-se fugas pontuais de classes DTO de infraestrutura na assinatura de serviços de domínio.
- **Concorrência Básica:** O processamento assíncrono é delegado com sucesso para

tarefas em segundo plano (Background Tasks), mas a leitura local de múltiplos ficheiros ainda recorre a mecanismos sequenciais simples sem paralelismo de coroutines.[3]

- **Cobertura de Testes Elevada:** A cobertura global de testes unitários situa-se entre 80% e 89%, com testes de integração funcionais mas sem cobrir caminhos de falha concorrente.
- **MCP e Neo4j Operacionais:** O servidor MCP funciona corretamente sobre stdio ou HTTP, mas ocorrem alucinações ocasionais em queries Cypher complexas de multi-hop, forçando o motor de recuperação a correr mais do que uma iteração de correção.[13]
- **Visualização Gráfica Funcional:** A consola renderiza o grafo de código, mas a representação é estática ou bidimensional simples, ou o cálculo de caminhos mínimos carece de reatividade instantânea no browser.[9]

Nível 3: Bom (14 a 16 Valores)

Implementações típicas de fim de curso que cumprem os requisitos de funcionamento básico, mas sem rigor arquitetural ou otimizações de performance:

- **Arquitetura Monolítica ou Acoplada:** O padrão de isolamento Ports & Adapters é quebrado por importações explícitas de bibliotecas web (FastAPI/Ktor) ou persistência no coração do domínio.[3]
- **Baixa Eficiência de Tokens:** O sistema não implementa poda de contexto consciente de critérios de aceitação (AC-Aware Pruning), enviando ficheiros de código completos e desestruturados para os agentes de IA, gerando desperdício e ultrapassando pontualmente a janela de tokens.[12]
- **Deduplicação Frágil:** A deduplicação de eventos concorrentes é efémera ou inexistente, confiando apenas num dicionário em memória RAM local que falha em ambientes com múltiplos contentores escalados.[3]
- **Cobertura de Testes Moderada:** A cobertura de testes unitários situa-se entre 60% e 79%, negligenciando testes de integração de rede ou de persistência com Neo4j.
- **Comentários Básicos:** O sistema publica comentários no Azure DevOps, mas sem formatação progressiva ou evidenciando apenas sugestões textuais curtas sem código alternativo concreto de correção.[3]

Nível 2: Suficiente (10 a 13 Valores)

Projetos mínimos que funcionam apenas em ambiente local controlado (sandbox) e falham nos requisitos não funcionais de segurança e latência:

- **Acoplamento Total:** Lógica de negócio escrita diretamente nos controladores da API Web, sem qualquer separação de camadas.
- **Inexistência de MCP ou Grafo:** O sistema limita-se a ler ficheiros de texto do disco e a enviá-los via prompt genérico a um modelo externo, ignorando o protocolo MCP e a base em grafos Neo4j.[12]
- **Timeout e Falhas de Rede:** O processamento ocorre de forma síncrona dentro da rota HTTP, provocando o timeout sistemático dos webhooks do Azure DevOps e publicações duplicadas repetidas.[3]

- **Testes Inexistentes ou Simbólicos:** Cobertura de testes inferior a 50%, com testes que apenas validam asserts tautológicos simples.
- **Soberania Comprometida:** Envio de código-fonte sem criptografia ou em canal HTTP não seguro para APIs públicas externas, violando as regras básicas de conformidade corporativa.[3]

Nível 1: Insuficiente (<10 Valores)

Projetos que não cumprem os requisitos mínimos de funcionamento ou apresentam falhas estruturais impeditivas:

- **Código Não Compila ou Não Inicia:** O backend Kotlin ou frontend TypeScript falham no build inicial.
- **Ausência de Entrega de Artefactos:** Falha na disponibilização de documentação arquitetural, repositórios de controlo de versão vazios ou ausência de Prova de Conceito funcional.
- **Vulnerabilidade Óbvia:** O gateway de webhooks permite a execução de chamadas arbitrárias sem qualquer tipo de validação de assinatura ou PATs expostos em texto plano no código.[3]

Trabalhos citados

1. licenciatura engenharia informática - ISEP, acesso a maio 27, 2026, https://www.isep.ipp.pt/files/isep_l_engenharia_informatica.pdf
2. software development process at LEI-ISEP - CDIO Initiative, acesso a maio 27, 2026, https://cdio.org/files/document/file/B4.3.pdf
3. Integração de Code Review Automatizado (3).pdf
4. Tecnologias da Informação e Comunicação no apoio ... - Jaime Villate, acesso a maio 27, 2026, https://villate.org/publications/Marques_2010_TICS_Ensino_Superior.pdf
5. ENHANCING STUDENTS' COMPETENCIES BY INTEGRATING MULTIPLE COURSE-UNITS ON SEMESTER PROJECTS - CDIO Initiative, acesso a maio 27, 2026, https://cdio.org/files/document/file/102_0.pdf
6. Spring Data Neo4j with Kotlin relation example, acesso a maio 27, 2026, https://community.neo4j.com/t/spring-data-neo4j-with-kotlin-relation-example/33678
7. spring-data · GitHub Topics, acesso a maio 27, 2026, https://github.com/topics/spring-data
8. Jovami - GitHub, acesso a maio 27, 2026, https://github.com/jovami
9. stakwork/stakgraph: A source code parser using treesitter ... - GitHub, acesso a maio 27, 2026, https://github.com/stakwork/stakgraph
10. GitHub - CodeGraphContext/CodeGraphContext: An MCP server plus a CLI tool that indexes local code into a graph database to provide context to AI assistants., acesso a maio 27, 2026, https://github.com/CodeGraphContext/CodeGraphContext
11. Redes de Computadores (RCOMP) - DEI-ISEP, acesso a maio 27, 2026,

https://www.dei.isep.ipp.pt/~asc/doc/RCOMP/2019-2020/PL10-RCOMP-2019-2020.pdf
12. codebase-memory-mcp | MCP Servers - LobeHub, acesso a maio 27, 2026, https://lobehub.com/mcp/deusdata-codebase-memory-mcp
13. Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP - arXiv, acesso a maio 27, 2026, https://arxiv.org/pdf/2603.27277
14. java-tree-sitter/README.md at master - GitHub, acesso a maio 27, 2026, https://github.com/tree-sitter/java-tree-sitter/blob/master/README.md
15. GitHub - jovami/isep-lapr5: ISEP 3rd year 1st semester Integrative Project 2023/2024, acesso a maio 27, 2026, https://github.com/jovami/isep-lapr5
16. Introducing Create Context Graph - Neo4j, acesso a maio 27, 2026, https://neo4j.com/blog/genai/introducing-create-context-graph/
17. I tested 4 codebase-to-AI tools on FastAPI (108k lines). Here are the token costs., acesso a maio 27, 2026, https://dev.to/thegdsks/i-tested-4-codebase-to-ai-tools-on-fastapi-108k-lines-here-are-the-token-costs-4bmc
18. Video: NODES AI 2026 - Build Intelligent AI Agents With Context Engineering - Neo4j, acesso a maio 27, 2026, https://neo4j.com/videos/nodes-ai-2026-build-intelligent-ai-agents-with-context-engineering/
19. Code property graph - Wikipedia, acesso a maio 27, 2026, https://en.wikipedia.org/wiki/Code_property_graph
20. Model Context Protocol (MCP) Integrations for the Neo4j Graph Database - Developer Guides, acesso a maio 27, 2026, https://neo4j.com/developer/genai-ecosystem/model-context-protocol-mcp/
21. CodexGraph: LLM-Driven Code Graphs - Emergent Mind, acesso a maio 27, 2026, https://www.emergentmind.com/topics/codexgraph
22. Spring Data Neo4j: A Guide to Graph Databases - codecentric AG, acesso a maio 27, 2026, https://www.codecentric.de/en/knowledge-hub/blog/spring-data-neo4j
23. Codebase-Memory: Tree-Sitter-Based Knowledge Graphs for LLM Code Exploration via MCP - arXiv, acesso a maio 27, 2026, https://arxiv.org/html/2603.27277v1
24. Java Tree-sitter - GitHub, acesso a maio 27, 2026, https://github.com/tree-sitter/java-tree-sitter
25. Package io.github.treesitter.jtreesitter, acesso a maio 27, 2026, https://tree-sitter.github.io/java-tree-sitter/
26. Describing a Property Graph Data Model - Neo4j, acesso a maio 27, 2026, https://neo4j.com/blog/developer/describing-property-graph-data-model/
27. GJordao12/ISEP-LAPR4 - Helpdesk aService · GitHub, acesso a maio 27, 2026, https://github.com/GJordao12/ISEP-LAPR4
28. Core concepts :: Spring Data Neo4j, acesso a maio 27, 2026, https://docs.spring.io/spring-data/neo4j/reference/data-commons/repositories/core-concepts.html
29. Releases · tree-sitter/java-tree-sitter - GitHub, acesso a maio 27, 2026,

https://github.com/tree-sitter/java-tree-sitter/releases
30. Hands On With Context Graphs And Neo4j, acesso a maio 27, 2026, https://neo4j.com/blog/agentic-ai/hands-on-with-context-graphs-and-neo4j/
31. Licenciatura em Engenharia Informática - ISEP - Instituto Superior de Engenharia do Porto, acesso a maio 27, 2026, https://www.isep.ipp.pt/Course/Course/26